GrowthAI: A Skill-Gap Driven Personal Growth and Time-Management Assistant with Rule-Based Adaptive Planning and Advisory Task-Completion Prediction

Abdul Kader Maahir S
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology and Advanced Studies
Chennai, India

P. Kishore
Department of Artificial Intelligence and Machine Learning
Vels Institute of Science, Technology and Advanced Studies
Chennai, India

V. P. Anitha
Department of Computer Science and Engineering
Vels Institute of Science, Technology and Advanced Studies
Chennai, India

Abstract— Most learners know what they want to become, but they often cannot tell which skills they are still missing, and the study plans they follow do not change when their days go badly. This paper describes GrowthAI, a web application that puts goal setting, skill assessment, skill-gap analysis, roadmap creation and daily task planning into a single workflow. After registering, a learner creates a goal and takes a timed multiple-choice test on the skills linked to that goal. The scores are turned into beginner, intermediate or advanced levels using fixed cut-offs, and the weaker skills are placed first in the roadmap. Daily tasks are then suggested from the learner's free time, energy check-ins and recent skipping, and each skipped task is saved together with a reason. The planner and the difficulty advice follow explicit rules. On top of the logged outcomes we added a small logistic-regression module that estimates whether a pending task will be finished. It is only advisory, it does not change the plan, and it runs only when the history holds enough completed and skipped tasks. The system uses React, FastAPI and PostgreSQL, and for security it relies on bcrypt hashing, signed JWT tokens, ownership checks and a server-timed assessment with attempt limits. We explain the design, report an end-to-end test with the Data Analyst goal, and list the limitations. An LLM-based coach and a weekly reflection are planned for later and are not part of what has been built.

***Keywords*— Personal growth assistant, skill assessment, skill-gap analysis, adaptive task planning, logistic regression, FastAPI.**

I. INTRODUCTION

Choosing a goal like "become a data analyst" is the easy part. Deciding what to study first, and how much to try on a tired day, is where most people get stuck. They usually write a timetable once and stop following it within a week. One-to-one guidance is known to improve learning far more than uniform classroom teaching [1], yet very few students can get it.

Two things bother us about the tools people use today. The first is that goals and skills live in different places: a quiz on one site, a roadmap on another, and task tracking somewhere else. The second is that plans do not learn from behaviour. When someone skips a task, the reason is almost never written down, so the next plan makes the same mistake again. Procrastination has been described as a failure of self-regulation [2], and the work on self-regulated learning puts planning, monitoring and reflection at the centre [3]. A timetable alone supports none of these three.

GrowthAI tries to connect these pieces. The flow runs from the goal to the required skills, the assessment, the skill gaps, the roadmap, the daily tasks and the behaviour tracking, and then back to the next plan. We have tried to be plain about what is finished and what is not. The personalization is mostly rule-based, the machine-learning part is a small advisory predictor, and the AI coach is future work. The contributions of this paper are:

1. One connected workflow, from goal to daily tasks, running end to end in a single web application.
2. A timed assessment controlled by the server, with deterministic shuffling of questions and options, attempt limits and tab-switch logging.
3. Rule-based skill levels and a roadmap that puts weaker skills first.
4. A rule-based daily planner that looks at free time, energy and skip history, and stores every skip reason for later analysis.
5. An advisory logistic-regression module that estimates the chance of completing a task from past outcomes, with features built to avoid data leakage.
6. A secure implementation using bcrypt, JWT and ownership checks, together with an honest list of its limitations.

II. RELATED WORK

A. Skill Modelling and Tutoring

Knowledge tracing estimates how well a learner has mastered a skill from the pattern of right and wrong answers [4]. Deep knowledge tracing later swapped the hand-built model for a recurrent network [5]. It can pick up richer patterns, but it needs large logs of student activity. GrowthAI does not follow mastery over time. It takes one assessment per attempt and applies simple cut-offs, which is less powerful but still works for a new user who has no history.

B. Procrastination and Self-Regulation

Steel's meta-analysis goes through the causes of procrastination and treats it as a self-regulatory failure [2]. Zimmerman describes self-regulated learners as people who set goals, watch their progress and change their strategies when needed [3]. We took two of these ideas into the product: the reason for each skipped task is recorded, and completion and skip rates are shown back to the learner.

C. Adaptive Recommendation and Prediction

Recommender systems often combine fixed rules with learned models [6]. Logistic regression is a standard choice for yes/no outcomes and is easy to interpret [7], and scikit-learn provides it [8]. We use it for a single narrow question, whether a pending task will be completed, because one learner generates only a small amount of data.

D. Secure Web Access

The bcrypt scheme was designed as an adaptive way of storing passwords [9], and JSON Web Tokens are a compact, signed way of carrying identity between client and server [10]. We use both, so the server never keeps a raw password and never trusts a user id sent by the client.

E. Open Problem

From what we have seen, quiz platforms test but not plan, planners schedule without knowing the learner's level, and habit trackers log behaviour without tying it to a goal. The problem we work on is joining these steps in one loop while being honest about how little data a new user has.

III. METHODOLOGY

Fig. 1 shows the overall architecture, and Fig. 2 shows the learner journey that the modules support.

A. System Architecture

The frontend is a React 19 single-page application built with Vite. It talks to a FastAPI backend through REST and JSON. The backend runs on Uvicorn and uses SQLAlchemy with plain SQL queries to read and write a PostgreSQL database hosted on Neon. scikit-learn is imported only when a prediction is requested, so the application still starts even if the machine-learning libraries cannot be loaded.

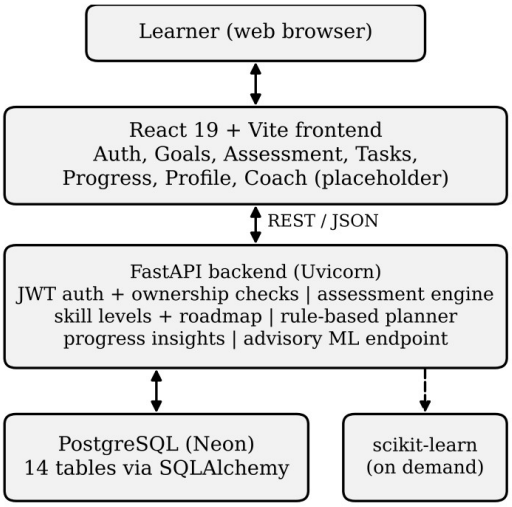


Fig. 1. Architecture of GrowthAI.

B. Authentication and Profile

Passwords are stored only as bcrypt hashes. A successful login returns a signed JWT with an expiry time, and every protected route reads the user from the token rather than from a value sent by the client. Goals, tasks, attempts and similar records are fetched with ownership checks. The profile holds age, occupation, daily available minutes, productive time, sleep hours, distraction level, procrastination level and challenges. It can be filled in after registration, so an unfinished onboarding never blocks a user.

C. Goals and Skill Mapping

A goal is linked to its required skills through a goal–skill table, and every link has an importance of high, medium or low. For the example goal "Become a Data Analyst", the mapped skills are Python, SQL, Statistics, Excel and Data Visualization. The seed data contains 13 skills in all. A goal with no predefined mapping simply has no linked skills until one is added.

D. Secure Skill Assessment

Questions are taken from a multiple-choice question bank and filtered by the skills of the active goal. When a learner starts an attempt, the server fixes a time budget from the number of questions, a configurable time per question and a short grace period, and it measures the elapsed time on its own clock. The order of the questions and the positions of the four options are shuffled with an HMAC of the attempt id under a server secret. The shuffle is recomputed on each request instead of being stored, which lets the server translate the letter a learner clicked back to the stored option. The correct answer is never sent to the browser, answers are submitted as one batch with per-question timing, and a repeated submission is ignored. A goal allows at most five completed attempts, and an unfinished attempt resumes with its original clock. Tab switches are counted and saved as an informational integrity signal, and copy, cut and right-click are disabled on the assessment page. This signal does not change the score.

E. Skill Levels, Skill Gaps and Roadmap

For each skill *s*, the score is the percentage of correct answers:

$$\text{score}(s) = 100 \times \text{correct}(s) / \text{total}(s) \quad (1)$$

A score below 40 gives the level beginner, a score from 40 up to, but not including, 70 gives intermediate, and a score of 70 or more gives advanced. This is a plain rule and not a learned model. Each skill becomes one roadmap step with a short recommendation. The steps are ordered so that weaker skills come first, and ties are broken by importance (high, medium, low).

F. Adaptive Daily Planning and Behaviour Tracking

The planner is rule-based. It looks at the roadmap priority, the available minutes from the profile, the latest energy check-in and the recent skip history. It suggests tasks with an estimated duration and a difficulty of easy, medium or hard. A learner can mark a task as completed or skipped, and a skip needs a reason. Every outcome goes into a task log. The progress page shows the total, completed and skipped tasks, the completion and skip rates, the most common skip reason, and a difficulty recommendation that comes from explicit conditions on energy, skipping and free time. When there is no history, the page says so instead of assuming values.

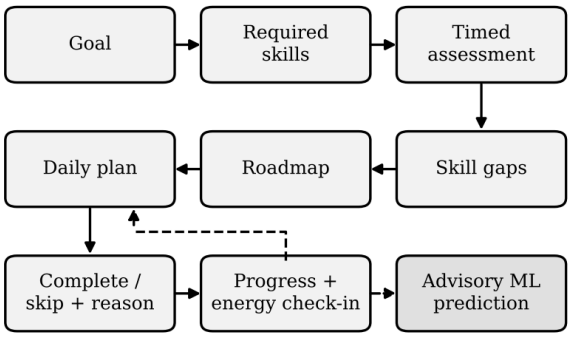


Fig. 2. Learner journey and feedback loop.

G. Advisory Task-Completion Prediction

For a pending task, the system estimates the chance that it will be completed. The features are listed in Table I. The training examples use the latest completed or skipped outcome of each task, and the historical rates and the recent skip count are calculated only from log entries recorded before that outcome, so no future information leaks into the features.

TABLE I. FEATURES USED BY THE PREDICTION MODULE

Feature	**Meaning**
Task duration	Estimated minutes of the task
Duration missing	1 if no duration was given
Available minutes	Daily minutes from the profile
Energy level	Latest energy check-in
Completion / skip rate	Earlier outcomes of the user
Recent skip count	Skips in the recent history
Difficulty	Easy = 1, medium = 2, hard = 3
Productive-time match	Task time vs. profile productive time

The model is a standard-scaled logistic regression:

$$p = \sigma(w^T x + b) = 1 / (1 + e^{\wedge -}(w^T x + b)) \quad (2)$$

A task with $p \geq 0.5$ is reported as “Likely to Complete”, and one below 0.5 as “At Risk”. A prediction is made only when enough logged outcomes exist (at least six) and both outcomes are present; otherwise the service answers “insufficient data”. When enough data is available, the module can also report holdout accuracy, precision, recall and ROC AUC. The output is advice only and does not change the task difficulty or the plan.

H. Database Design

The database has 14 tables. Identity and profile: users, user_profiles. Goals and skills: goals, skills, goal_skills. Assessment: assessment_questions, assessment_attempts, assessment_answers, skill_assessments. Planning and behaviour: tasks, task_logs, energy_checkins, progress. The coach_conversations table is kept for the planned coach and is not used yet.

IV. RESULTS AND ANALYSIS

A. Setting

The system was run on a Windows laptop with the Vite development server (port 5173), the FastAPI backend (port 8000) and a Neon-hosted PostgreSQL database. The seed data holds 13 skills and 15 assessment questions, and seven of those belong to the Data Analyst skills. The tests below are functional checks made by the developers, not a user study.

B. End-to-End Functional Test

A demo user registered, logged in and created the goal “Become a Data Analyst”. The five mapped skills appeared, and the assessment served the seven available questions in a shuffled order with a countdown timer. Table II shows the result of one test attempt. The overall score was 28.57% (2 of 7 correct). The levels follow the thresholds of Section III-E: Python scored 50% and is intermediate, Excel scored 100% and is advanced, and the other three skills scored 0% and are beginner. After submission, the roadmap appeared on the same result screen with one step and one recommendation for each skill.

TABLE II. RESULT OF ONE TEST ASSESSMENT (DATA ANALYST)

Skill	**Correct**	**Score**	**Level**
Python	1 / 2	50%	Intermediate
SQL	0 / 2	0%	Beginner
Statistics	0 / 1	0%	Beginner
Excel	1 / 1	100%	Advanced
Data Visualization	0 / 1	0%	Beginner
Overall	**2 / 7**	**28.57%**	–

C. Tasks, Skipping and Progress

Marking a task as completed or skipped updated the task log and the progress totals, and a skip could not be saved without a reason. After a low-energy check-in, the rule-based advice changed in the expected direction, which is the point of using the learner's condition in planning. These checks show that the pieces work together, but they do not show whether learners actually improve.

D. Status of the Prediction Module

The prediction needs at least six logged outcomes with both classes, and the test accounts created so far do not have enough such data. For that reason, accuracy, precision, recall and ROC AUC values are not reported here [ADD: metrics after collecting more task history]. During development, a Windows application-control policy on one laptop blocked part of the scientific Python libraries. The module was written to return an "unavailable" message in that case, so the rest of the application keeps working.

TABLE III. CURRENT STATUS OF THE MAIN FEATURES

Feature	**Status**
JWT login, ownership checks	Implemented
Goal–skill mapping	Implemented
Timed, shuffled, limited assessment	Implemented
Skill levels and roadmap	Implemented (rule-based)
Daily planner, skip reasons	Implemented (rule-based)
Progress insights	Implemented
Task-completion prediction	Implemented (advisory)
AI coach, weekly reflection	Planned
Automated tests	Pending

E. Limitations

The question bank is small, so questions repeat across attempts. Personalization is rule-based and not learned. The prediction is advisory and depends on how much history a learner has. Goal-to-skill mapping is predefined only for known goals. No large-scale evaluation or user study has been done, so the results show that the system works, not that it improves learning outcomes.

V. CONCLUSION

We presented GrowthAI, a web application that joins goal setting, a secure skill assessment, skill-gap analysis, a roadmap, adaptive daily planning and behaviour tracking in one loop. The planner and the difficulty advice are rule-based, and a logistic-regression module gives an advisory estimate of task completion once enough data exists. A working test with the Data Analyst goal shows that the modules fit together. This is a first prototype, limited by a small question bank, little task history and the lack of a user study.

In future work we plan to enlarge the question bank, including validated AI-generated questions, to evaluate the prediction module on real task histories, and to test whether learners who follow the roadmap complete more tasks. After that, we plan to add an LLM-based personal coach and a weekly reflection, and to look at a learned planning policy. These coach features are not part of the current system.

REFERENCES

[1] B. S. Bloom, 'The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring', Educational Researcher, vol. 13, no. 6, pp. 4–16, 1984.

[2] P. Steel, 'The nature of procrastination: A meta-analytic and theoretical review of quintessential self-regulatory failure', Psychological Bulletin, vol. 133, no. 1, pp. 65–94, 2007.

[3] B. J. Zimmerman, 'Becoming a self-regulated learner: An overview', Theory Into Practice, vol. 41, no. 2, pp. 64–70, 2002.

[4] A. T. Corbett and J. R. Anderson, 'Knowledge tracing: Modeling the acquisition of procedural knowledge', User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253–278, 1994.

[5] C. Piech et al., 'Deep Knowledge Tracing', in Advances in Neural Information Processing Systems, 2015, vol. 28.

[6] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 2nd ed. New York, NY, USA: Springer, 2015.

[7] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, Applied Logistic Regression, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[8] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python', Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] N. Provos and D. Mazières, 'A Future-Adaptable Password Scheme', in Proceedings of the USENIX Annual Technical Conference, FREENIX Track, 1999, pp. 81–91.

[10] M. Jones, J. Bradley, and N. Sakimura, 'JSON Web Token (JWT)', RFC 7519, Internet Engineering Task Force, May 2015.